

Documentación Proyecto aplicación IOT



Implantación de Aplicaciones Web
Juan Molina
2º ASIR

Índice

1. PRIMERA PARTE: RECOLECTOR DE DATOS DE LA API.....	4
2. SEGUNDA PARTE: TRASPASO DE DATOS PROVEEDOR/CLIENTE CON MOSQUITTO.....	5
3. TERCERA PARTE: TELEGRAF, INFLUXDB.....	8
4. CUARTA PARTE: ENLACE DE PYTHON A MOSQUITTO.....	10
5. QUINTA PARTE: GRAFANA.....	13
6. RESULTADO FINAL.....	19

PRIMERA PARTE: RECOLECTOR DE DATOS DE LA API

1. Crear Carpeta del proyecto

```
mkdir Proyecto_IOT_VLC_Bicis
```

2. Crear el collector

```
mkdir Proyecto_IOT_VLC_Bicis/collector
```

3. Crear entorno virtual para instalar paquetes y situándonos en la carpeta raíz del proyecto activamos el entorno virtual

```
python3 -m venv .venv  
source .venv/bin/activate
```

4. Instalamos requests para que python realice peticiones HTTP en nuestro proyecto

```
pip install requests  
pip list #Esto lista los paquetes instalados
```

5. Dentro de la carpeta collector creamos requirements.txt para dejar las dependencias

```
nano collector/requirements.txt  
#Dentro de requirements.txt se escribe lo siguiente y se guarda:  
requests  
paho-mqtt
```

6. Crear el colector mediante el siguiente script de python

```
nano collector/collector.py  
  
#Dentro de collector.py se escribe el siguiente script:  
  
import requests  
import time  
  
API_URL = "http://api.citybik.es/v2/networks/valenbisi"  
  
def main():
```

```
estaciones = requests.get(API_URL,
timeout=10).json()["network"]["stations"]

while True:
    for e in estaciones:
        print(f"Estación: {e['name']}")
        print(f"  Bicicletas libres: {e['free_bikes']}")
        print(f"  Huecos libres: {e['empty_slots']}")
        print()
        time.sleep(3)

if __name__ == "__main__":
    main()
```

7. Verificar el correcto funcionamiento del programa

```
python3 collector/collector.py
```

SEGUNDA PARTE: TRASPASO DE DATOS PROVEEDOR/CLIENTE CON MOSQUITTO

1. Crear carpeta mosquito

```
mkdir -p mosquito/config
```

2. Crear archivo de configuración en ./mosquito/config y escribir la siguiente configuración

```
nano mosquito.conf
#Dentro del archivo
listener 1883
allow_anonymous true
```

3. Crear el archivo docker-compose.yml para configurar mosquito

```
nano docker-compose.yml
```

```
#Dentro del archivo
```

```
services:

  mosquitto:
    image: eclipse-mosquitto:2
    container_name: valenbisi-mosquitto
    ports:
      - "1883:1883"
    volumes:
      -
        ./mosquitto/config/mosquitto.conf:/mosquitto/config/mosquitto.conf
```

4. Levantar el servicio mosquito

```
docker compose up -d mosquitto
```

5. Comprobar que la arquitectura publisher/subscriber funciona (Esto ya que en este punto el script de python no está enlazado con mosquito). Con esto verificamos que en el subscriber llegan los mensajes.

```
#Terminal 1:Subscriber
cd Proyecto_IOT_VLC_Bicis
docker exec -it valenbisi-mosquitto mosquitto_sub -h localhost -t '#' -v

#Terminal 2: Publisher
cd Proyecto_IOT_VLC_Bicis
docker exec valenbisi-mosquitto mosquitto_pub -h localhost -t prueba -m
"HOLA"
```

6. Instalamos el paquete en nuestro entorno virtual

```
pip install paho-mqtt
```

7. Modificamos el código del colector para que el script de Python se pueda conectar a mosquitto.

```
nano collector.py

#Dentro modificamos el código

import requests
import time
import json
import paho.mqtt.client as mqtt

API_URL = "http://api.citybik.es/v2/networks/valenbisi"

MQTT_BROKER = "localhost"
MQTT_PORT = 1883
MQTT_TOPIC = "valenbisi/stations"

def main():
    cliente = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
    cliente.connect(MQTT_BROKER, MQTT_PORT)
    cliente.loop_start()

    while True:
        estaciones = requests.get(API_URL,
        timeout=10).json()["network"]["stations"][:100]

        for e in estaciones:

            mensaje = {
                "station": e["name"],
                "free_bikes": e["free_bikes"],
                "empty_slots": e["empty_slots"],
                "latitude": e["latitude"],
                "longitude": e["longitude"]
            }

            cliente.publish(MQTT_TOPIC, json.dumps(mensaje))

            print(f"Estación: {e['name']}\n Bicicletas libres:
            {e['free_bikes']}\n Huecos libres: {e['empty_slots']}\n")

            time.sleep(3)
```

```
if __name__ == "__main__":  
    main()
```

8. Comprobamos que la conexión Python/Mosquitto funciona, para ello ejecutamos en dos terminales ejecutamos los siguientes comandos_

```
#Terminal 1:Subscriber  
cd Proyecto_IOT_VLC_Bicis  
docker exec -it valenbisi-mosquitto mosquitto_sub -h localhost -t  
valenbisi/stations  
  
#Terminal 2:Publisher  
cd Proyecto_IOT_VLC_Bicis  
python3 collector/collector.py
```

TERCERA PARTE: TELEGRAF, INFLUXDB

1. Creamos la carpeta de telegraf

```
mkdir telegraf
```

2. Creamos un fichero de configuración con los siguientes parámetros para que telegraf sea intermediario con mosquitto y la base de datos de InfluxDB que recogerá los datos.

```
nano telegraf/telegraf.conf  
  
#Dentro del fichero  
  
[agent]  
    interval = "10s"  
    flush_interval = "10s"  
  
[[inputs.mqtt_consumer]]
```

```
servers = ["tcp://mosquitto:1883"]

topics = [
  "valenbisi/stations"
]

data_format = "json"

tag_keys = [
  "station"
]

[[outputs.influxdb_v2]]
  urls = ["http://influxdb:8086"]

  token = "valenbisi-token"

  organization = "valenbisi"

  bucket = "valenbisi"
```

3. En el docker-compose.yml, añadimos el servicio de InfluxDB y telegraf

```
nano docker-compose.yml

#Dentro del fichero
services:

  mosquitto:
    image: eclipse-mosquitto:2
    container_name: valenbisi-mosquitto
    ports:
      - "1883:1883"
    volumes:
      -
        ./mosquitto/config/mosquitto.conf:/mosquitto/config/mosquitto.conf

  influxdb:
    image: influxdb:2
    container_name: valenbisi-influxdb
    ports:
      - "8086:8086"
    volumes:
```

```
- ./influxdb:/var/lib/influxdb2
environment:
  DOCKER_INFLUXDB_INIT_MODE: setup
  DOCKER_INFLUXDB_INIT_USERNAME: admin
  DOCKER_INFLUXDB_INIT_PASSWORD: admin12345
  DOCKER_INFLUXDB_INIT_ORG: valenbisi
  DOCKER_INFLUXDB_INIT_BUCKET: valenbisi
  DOCKER_INFLUXDB_INIT_ADMIN_TOKEN: valenbisi-token

telegraf:
  image: telegraf:1.36
  container_name: valenbisi-telegraf
  depends_on:
    - mosquitto
    - influxdb
  volumes:
    - ./telegraf/telegraf.conf:/etc/telegraf/telegraf.conf:ro
```

4. Levantar los servicio de servicios y comprobar los logs si se llegan a producir errores

```
docker compose up -d
docker compose logs
```

CUARTA PARTE: ENLACE DE PYTHON A MOSQUITTO

1. Creamos un dockerfile en collector, con el fin de crear una aplicación que docker pueda manejar

```
nano collector/Dockerfile
```

2. En el dockerfile añadimos lo siguiente:

```
FROM python:3.13-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY collector.py .

CMD ["python", "collector.py"]
```

3. En el script de colector, cambiamos el broker de “localhost” a “mosquitto”

```
MQTT_BROKER = "mosquitto"
```

4. Añadimos la app del collector al docker-compose.yml

```
nano docker-compose.yml

#Dentro del fichero
services:

  mosquitto:
    image: eclipse-mosquitto:2
    container_name: valenbisi-mosquitto
    ports:
      - "1883:1883"
    volumes:
      -
        ./mosquitto/config/mosquitto.conf:/mosquitto/config/mosquitto.conf

  influxdb:
    image: influxdb:2
    container_name: valenbisi-influxdb
    ports:
      - "8086:8086"
    volumes:
      - ./influxdb:/var/lib/influxdb2
    environment:
      DOCKER_INFLUXDB_INIT_MODE: setup
      DOCKER_INFLUXDB_INIT_USERNAME: admin
```

```
DOCKER_INFLUXDB_INIT_PASSWORD: admin12345
DOCKER_INFLUXDB_INIT_ORG: valenbisi
DOCKER_INFLUXDB_INIT_BUCKET: valenbisi
DOCKER_INFLUXDB_INIT_ADMIN_TOKEN: valenbisi-token

telegraf:
  image: telegraf:1.36
  container_name: valenbisi-telegraf
  depends_on:
    - mosquitto
    - influxdb
  volumes:
    - ./telegraf/telegraf.conf:/etc/telegraf/telegraf.conf:ro

collector:
  build: ./collector
  container_name: valenbisi-collector
  depends_on:
    - mosquitto
```

5. Creamos la aplicación y levantamos todo

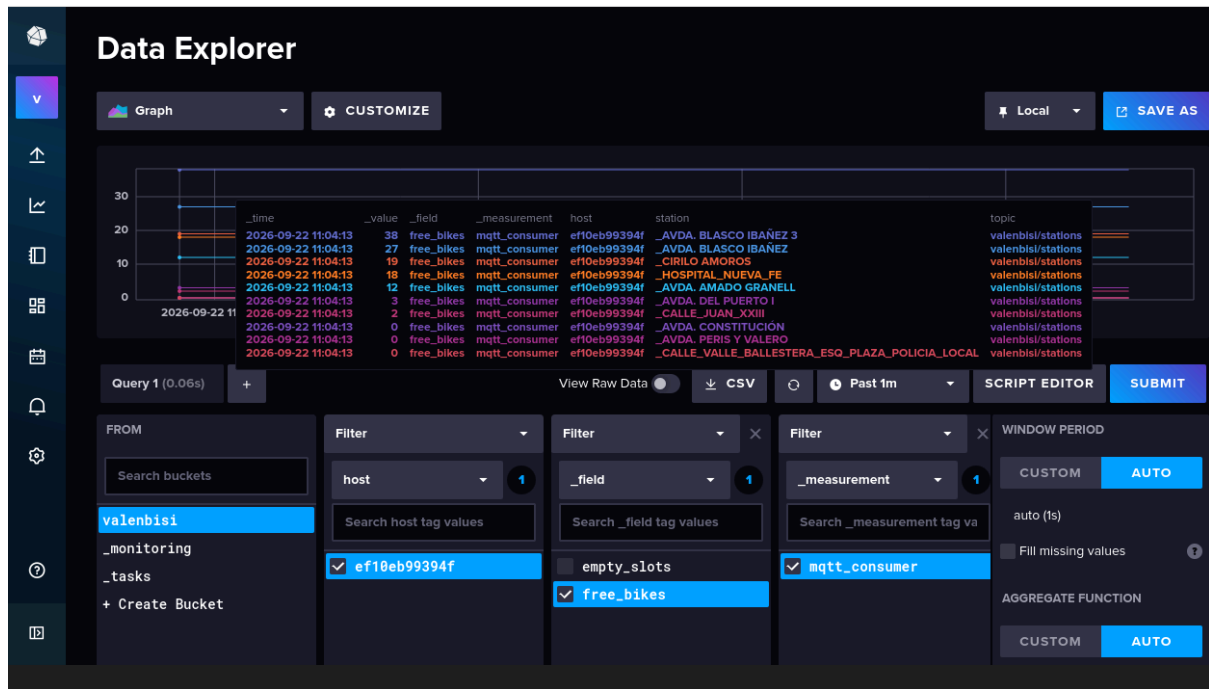
```
docker compose build collector
docker compose up -d
```

6. Se comprueba que todo esta en correcto funcionamiento

```
docker compose logs collector
docker compose logs telegraf
docker compose logs influxdb
```

En el caso de influxdb entramos por <http://localhost:8086>

Dentro comprobamos que tenemos los datos que le llegan a influxdb a través de telegraf:



QUINTA PARTE: GRAFANA

1. Creamos la carpeta del servicio grafana con las siguientes carpetas

```
mkdir -p grafana/provisioning/datasources
mkdir -p grafana/provisioning/dashboards
mkdir -p grafana/dashboards
```

2. Añadimos grafana al compose

```
nano docker-compose.yml

#Dentro del fichero

services:

  mosquitto:
    image: eclipse-mosquitto:2
    container_name: valenbisi-mosquitto
    ports:
      - "1883:1883"
```

```
volumes:
  -
./mosquitto/config/mosquitto.conf:/mosquitto/config/mosquitto.conf

influxdb:
  image: influxdb:2
  container_name: valenbisi-influxdb
  ports:
    - "8086:8086"
  volumes:
    - ./influxdb:/var/lib/influxdb2
  environment:
    DOCKER_INFLUXDB_INIT_MODE: setup
    DOCKER_INFLUXDB_INIT_USERNAME: admin
    DOCKER_INFLUXDB_INIT_PASSWORD: admin12345
    DOCKER_INFLUXDB_INIT_ORG: valenbisi
    DOCKER_INFLUXDB_INIT_BUCKET: valenbisi
    DOCKER_INFLUXDB_INIT_ADMIN_TOKEN: valenbisi-token

telegraf:
  image: telegraf:1.36
  container_name: valenbisi-telegraf
  depends_on:
    - mosquitto
    - influxdb
  volumes:
    - ./telegraf/telegraf.conf:/etc/telegraf/telegraf.conf:ro

collector:
  build: ./collector
  container_name: valenbisi-collector
  depends_on:
    - mosquitto

grafana:
  image: grafana/grafana:latest
  container_name: valenbisi-grafana
  depends_on:
    - influxdb
  ports:
    - "3000:3000"
  volumes:
    - ./grafana:/var/lib/grafana
    - ./grafana/provisioning:/etc/grafana/provisioning
```

3. Levantamos grafana

```
docker compose up -d grafana
```

#Si presenta errores como por ejemplo que no se pudo crear la carpeta debido a problemas de permisos le damos permisos a grafana para poner acceder a las carpetas

```
sudo chown -R 472:472 grafana
```

4. Comprobamos que los 5 servicios están montados

```
docker ps
```

```
(.venv) juan@machine:~/Escritorio/Proyecto_IOT_VLC_Bicis$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
dbab52f6978f   grafana/grafana:latest             "/run.sh"               5 minutes ago Up 2 seconds  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   valenbisi-grafana
589213fe3c5c   proyecto_iot_vlc_bicis-collector   "python collector.py"   29 minutes ago Up 29 minutes  8092/udp, 8125/udp, 8094/tcp           valenbisi-collector
ef18eb99394f   telegraf:1.36                     "/entrypoint.sh tele_" 41 minutes ago Up 41 minutes  0.0.0.0:8086->8086/tcp, [::]:8086->8086/tcp   valenbisi-telegraf
564bf9d25cc2   influxdb:2                         "/entrypoint.sh infl_" 47 minutes ago Up 47 minutes  0.0.0.0:1883->1883/tcp, [::]:1883->1883/tcp   valenbisi-influxdb
1091e08ced78   eclipse-mosquitto:2               "/docker-entrypoint..." 2 hours ago    Up 2 hours    0.0.0.0:1883->1883/tcp, [::]:1883->1883/tcp   valenbisi-mosquitto
```

PARTE FINAL: CONFIGURACIÓN GUID GRAFANA

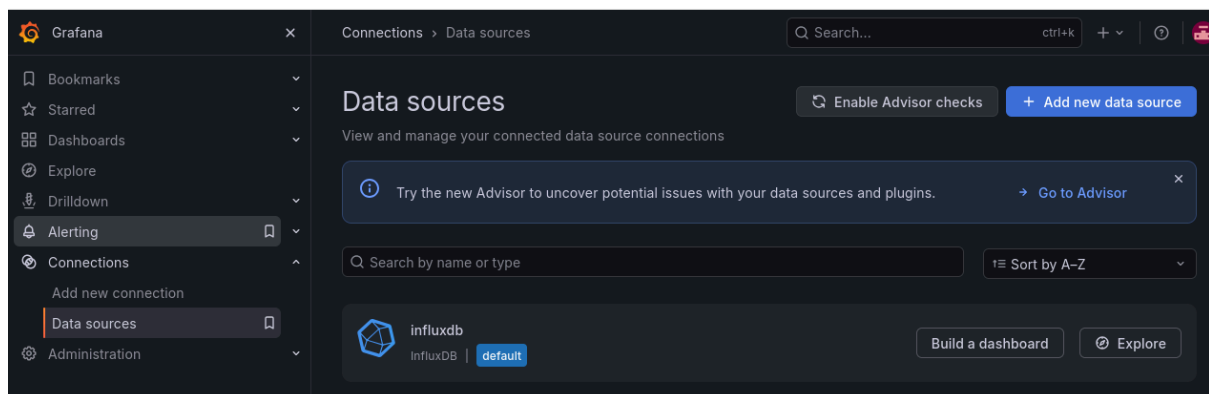
1. Accedemos a grafana mediante localhost y el puerto que tiene asignado

```
http://localhost:3000
```

2. Iniciamos sesión con las credenciales asignadas anteriormente

3. Conectamos InfluxDB con grafana, Accedemos a la siguiente ruta:

```
Conexiones -> Data sources -> Add data source -> InfluxDB
```



4. Dentro configuramos la URL para que se pueda conectar Influxdb que contiene la base de datos de la API a Influxdb web

URL
Your access method is *Server*, this means the URL needs to be accessible from the grafana backend/server.

`http://influxdb:8086`

5. Se rellena el resto de parámetros con los datos de la BBDD y terminamos de vincular los datos a la web

Query language

Flux

InfluxDB Details

Organization	valenbisi
Token	●●●●●●●●●●●●●●●●
Default Bucket	valenbisi
Min time interval ⓘ	10s
Max series ⓘ	1000

6. Para crear los dashboards necesitamos varios scripts en FLUX para que arroje los datos que se necesitan, necesitamos 8 scripts:

#Total bicicletas libres

```
from(bucket: "valenbisi")
  |> range(start: -1h)
  |> filter(fn: (r) => r._field == "free_bikes")
  |> group(columns: ["station"])
  |> last()
  |> group()
  |> sum(column: "_value")
```

#Total bicis ocupadas

```
from(bucket: "valenbisi")
  |> range(start: -1h)
  |> filter(fn: (r) => r["_field"] == "empty_slots")
  |> group()
  |> distinct(column: "station")
  |> count()
```

#Total de Estaciones

```
from(bucket: "valenbisi")
  |> range(start: -1h)
  |> filter(fn: (r) => r._field == "empty_slots")
  |> group(columns: ["station"])
  |> last()
  |> group()
  |> sum(column: "_value")
```

#Evolución de bicis libres (Gráfico)

```
from(bucket: "valenbisi")
  |> range(start: -6h)
  |> filter(fn: (r) => r._field == "free_bikes")
```

#Mapa de las estaciones

```
from(bucket: "valenbisi")
  |> range(start: -1h)
  |> filter(fn: (r) =>
    r._field == "latitude" or
    r._field == "longitude"
  )
  |> pivot(
    rowKey: ["station", "_time"],
    columnKey: ["_field"],
    valueColumn: "_value"
  )
  |> group()
  |> keep(columns: ["station", "latitude", "longitude"])
```

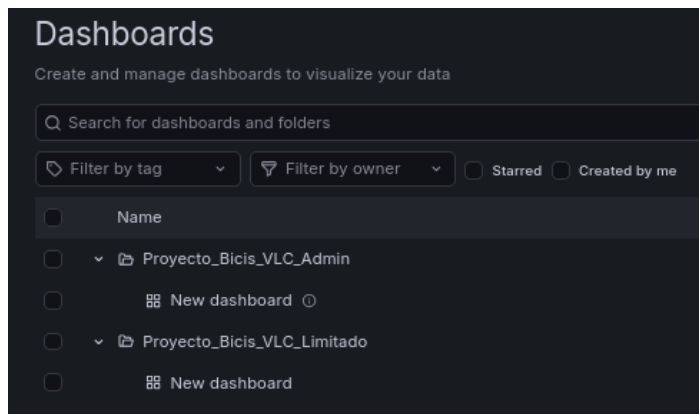
#Total bicis libres por estación

```
from(bucket: "valenbisi")
  |> range(start: -1h)
  |> filter(fn: (r) => r._field == "free_bikes")
  |> group(columns: ["station"])
  |> last()
  |> keep(columns: ["station", "_value"])
```

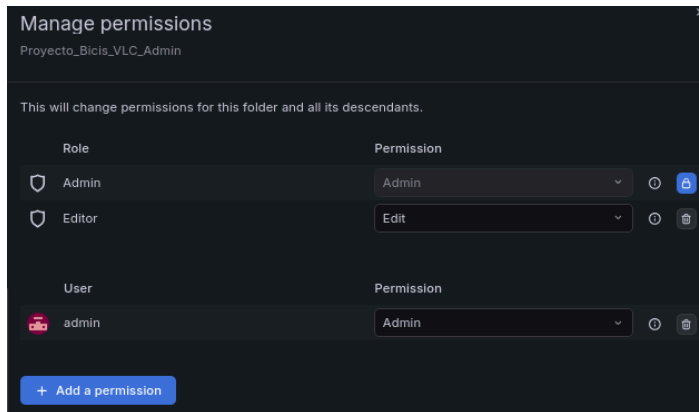
#Total bicis ocupadas por estación

```
from(bucket: "valenbisi")
  |> range(start: -1h)
  |> filter(fn: (r) => r._field == "empty_slots")
  |> group(columns: ["station"])
  |> last()
  |> keep(columns: ["station", "_value"])
```

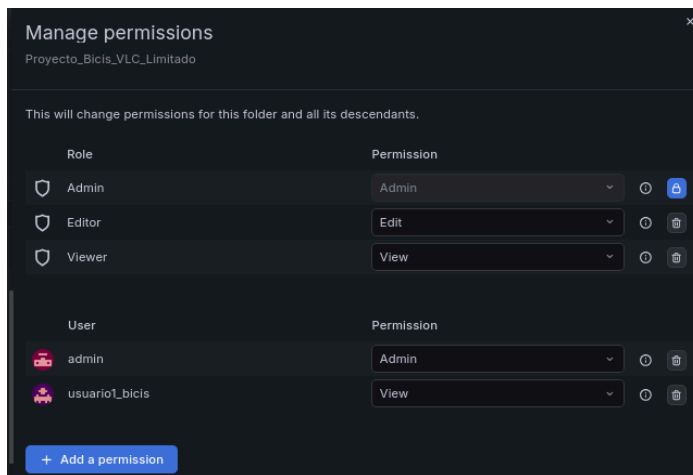
7. Por último se configuran los permisos, para ello creamos dos carpetas con 2 dashboards. Cada uno de los dashboards tendrá sus respectivos permisos para inspeccionarlos dependiendo si es la versión ampliada o limitada.



- Versión ampliada



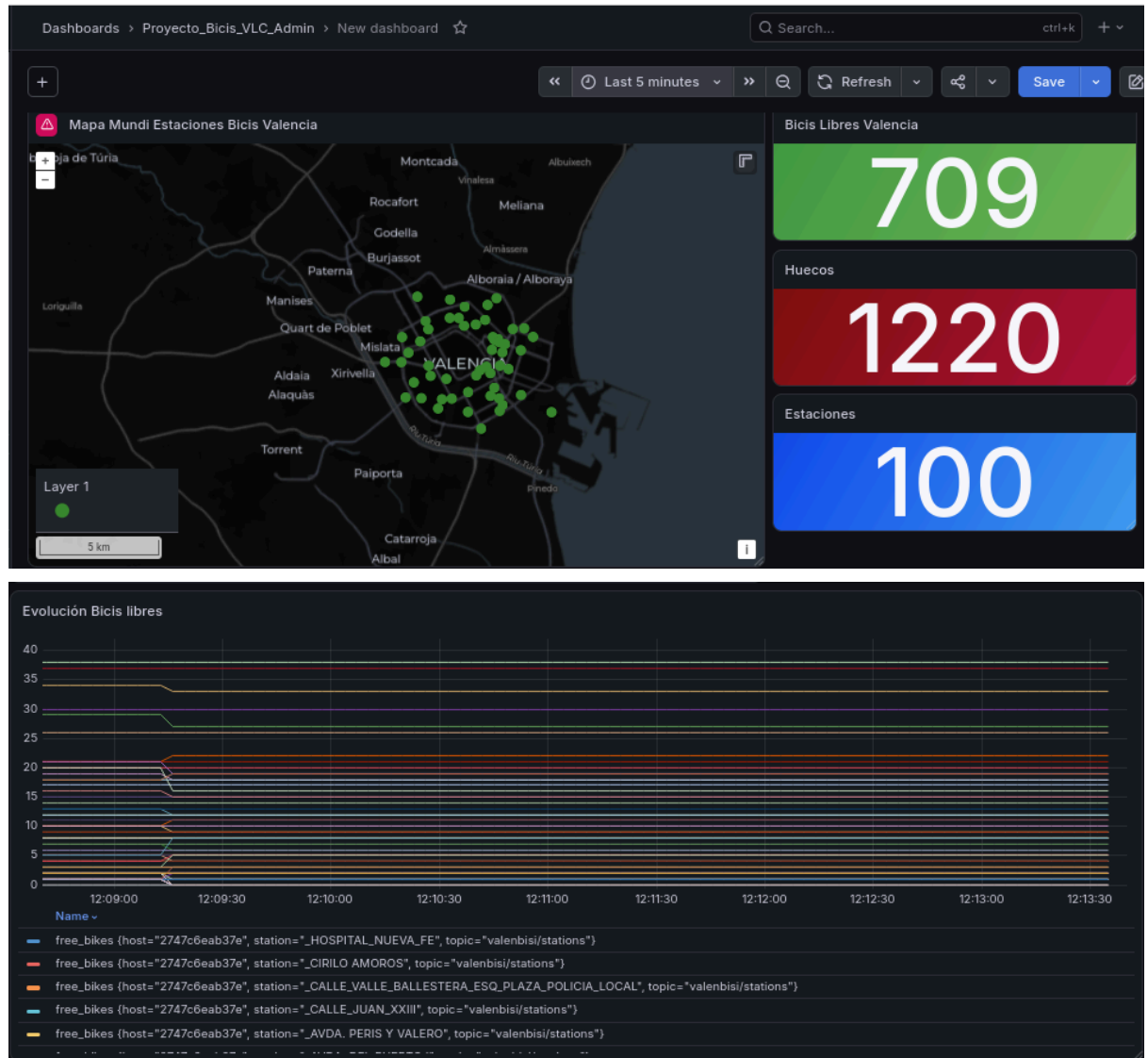
- Versión limitada

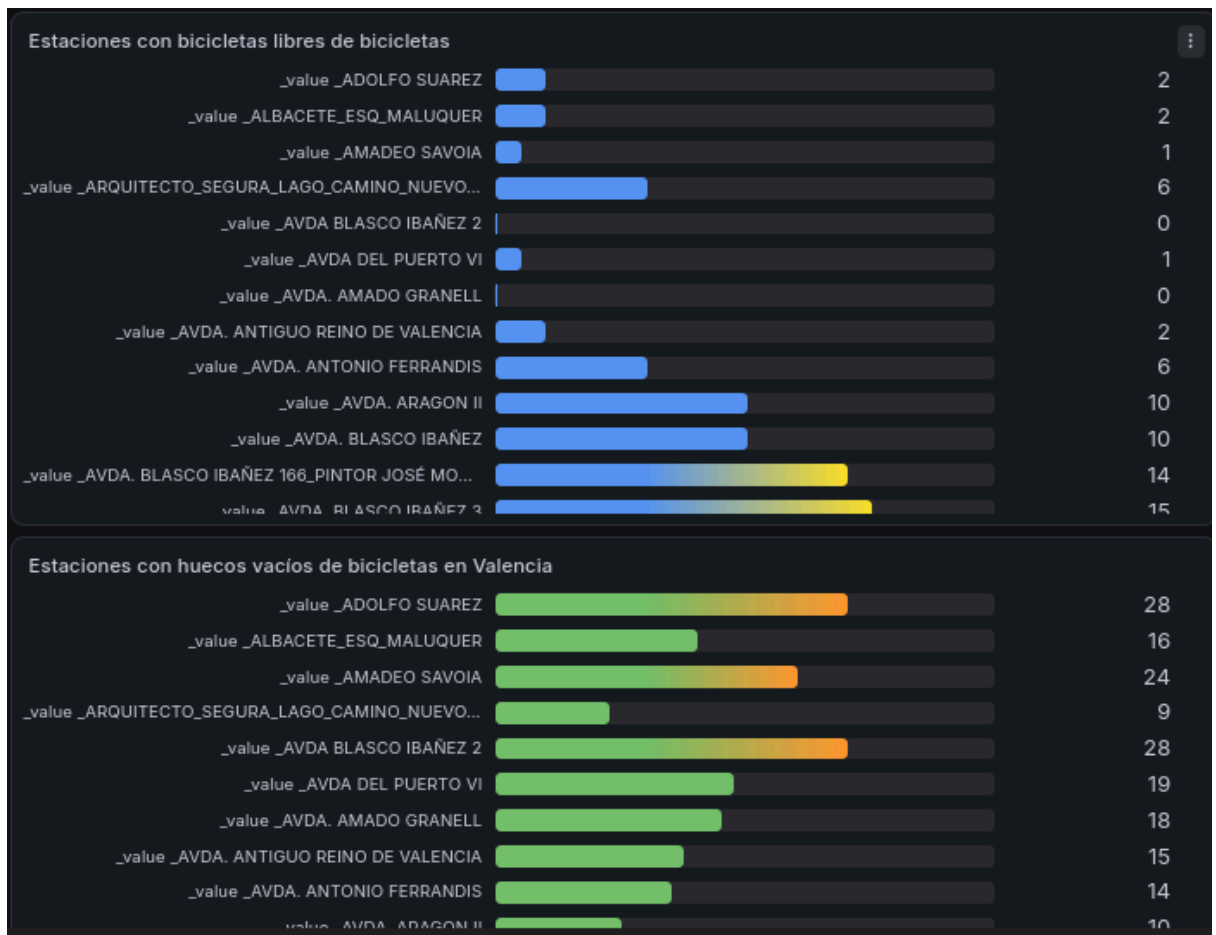


Con esto se consigue que un usuario promedio no tenga acceso a la versión ampliada del dashboard.

RESULTADO FINAL

- Versión ampliada





- Versión limitada

